Manual del Desarrollador

Sistema de Gestion Empresarial — Suelo Sur SAS

Version: 1.0
Fecha: Abril 2026
Stack: Node.js · Express 4.16 · EJS · Supabase (PostgreSQL)

Indice

1. Introduccion y Arquitectura General
2. Estructura del Proyecto
3. Instalacion y Configuracion
4. Pipeline de Middleware (app.js)
5. Sistema de Rutas
6. Controladores
7. Capa de Datos (Store)
8. Biblioteca de Utilidades (lib/)
9. Middleware de Autenticacion
10. Vistas (EJS)
11. Assets Estaticos (CSS/SCSS/JS)
12. Base de Datos — Tablas y Relaciones
13. Flujos de Negocio
14. Despliegue (Render.com)
15. Seguridad

---

1. Introduccion y Arquitectura General

El sistema es una aplicacion web MVC server-side que gestiona alquileres de contenedores, ventas de cantera, viajes programados, stock de productos, clientes con cuenta corriente y transacciones financieras para la empresa Suelo Sur SAS.

Diagrama de Arquitectura

```
Browser (EJS renderizado)
  |
  v
bin/www  (servidor HTTP, puerto 3000)
  |
  v
app.js  (Express + pipeline de middlewares)
  |
  +-- Rutas PUBLICAS ----------- login.controller
  |
  +-- requireAuth (guard de sesion)
  |
  +-- Rutas PROTEGIDAS --------- controllers/
  |     home.controller
  |     ventas.controller
  |     alquiler.controller
  |     contenedores.controller
  |     stock.controller
  |     clientes.controller
  |     transacciones.controller
  |
```

```
  +-- Rutas ADMIN (requireAdmin) - config.controller
        |
        v
      store/  (capa de queries Supabase)
        |
        v
      lib/supabase.js  (cliente Supabase singleton)
        |
        v
      Supabase Cloud PostgreSQL
```

Modelo de autenticacion: Sesiones con `express-session` (TTL 8 horas). La sesion almacena `{ id, user, rol }`. Roles: `admin` y `operador`.

---

2. Estructura del Proyecto

```
SeminarioGestionEmpresarial/
+-- .env                        Variables de entorno (secretas, no versionadas)
+-- .gitignore
+-- package.json
+-- render.yaml                 Configuracion de deploy en Render.com
+-- app.js                      Aplicacion Express principal
+-- bin/
|   +-- www                     Entry point del servidor HTTP
+-- controllers/
|   +-- alquiler.controller.js
|   +-- clientes.controller.js
|   +-- config.controller.js
|   +-- contenedores.controller.js
|   +-- home.controller.js
|   +-- login.controller.js
|   +-- stock.controller.js
|   +-- transacciones.controller.js
|   +-- ventas.controller.js
+-- lib/
|   +-- password.js             Utilidades bcrypt
|   +-- supabase.js             Cliente Supabase singleton
+-- middleware/
|   +-- requireAdmin.js
|   +-- requireAuth.js
+-- services/                   (legacy, no se usa actualmente)
|   +-- alquilerService.js
|   +-- contenedorService.js
+-- store/
|   +-- dbAlquiler.js
|   +-- dbClientes.js
|   +-- dbContenedor.js
|   +-- dbStock.js
|   +-- dbTransacciones.js
|   +-- dbUsers.js
|   +-- dbViajes.js
+-- views/
|   +-- error.ejs
|   +-- home.ejs
|   +-- alquileres/    (5 vistas)
```

```
|   +-- clientes/      (5 vistas)
|   +-- configuraciones/ (2 vistas)
|   +-- contenedores/  (3 vistas)
|   +-- login/         (2 vistas)
|   +-- partials/      (8 parciales reutilizables)
|   +-- stock/         (4 vistas)
|   +-- transacciones/ (2 vistas)
|   +-- ventas/        (5 vistas)
+-- public/
    +-- css/styles.css
    +-- img/suelosurjpg.jpg
    +-- javascript/    (8 archivos JS cliente)
    +-- scss/          (18 parciales + style.scss)
```

---

3. Instalacion y Configuracion

Requisitos previos

- Node.js >= 18
- npm
- Cuenta en Supabase con el proyecto creado

Pasos

```
# 1. Clonar el repositorio
git clone <url-del-repo>
cd SeminarioGestionEmpresarial

# 2. Instalar dependencias
npm install

# 3. Crear archivo .env en la raiz
```

El archivo `.env` debe contener:

```
SUPABASE_URL=https://tu-proyecto.supabase.co
SUPABASE_KEY=tu-anon-key-o-service-key
```

Ejecutar en desarrollo

```
npm run dev
```

Este comando usa `concurrently` para ejecutar en paralelo:

- `nodemon ./bin/www` — servidor con auto-reload
- `sass --watch public/scss/style.scss:public/css/styles.css` — compilacion SCSS en tiempo real

Ejecutar en produccion

```
npm start
```

Ejecuta `node ./bin/www`. El puerto se toma de la variable de entorno `PORT` (default: 3000).

---

4. Pipeline de Middleware (app.js)

El archivo `app.js` configura la aplicacion Express en el siguiente orden:

Orden	Middleware	Descripcion
1	`dotenv.config()`	Carga variables de `.env`
2	`morgan('dev')`	Log de requests HTTP en consola
3	`express.json()`	Parseo de body JSON
4	`express.urlencoded({ extended: false })`	Parseo de formularios
5	`cookieParser()`	Parseo de cookies
6	`express-session`	Sesiones con cookie de 8h
7	`express.static('public')`	Sirve archivos estaticos
8	**Globals custom**	Adjunta `formatFecha` y `sessionUser` a `res.locals`

Configuracion de sesion

```
app.use(session({
    secret: 'gestion-empresarial-secret',
    resave: false,
    saveUninitialized: false,
    cookie: { maxAge: 8 * 60 * 60 * 1000 }  // 8 horas
}));
```

Funcion global formatFecha

Disponible en todas las vistas EJS como `formatFecha(dateStr)`. Convierte fechas ISO a formato `DD-MM-YYYY`.

---

5. Sistema de Rutas

Todas las rutas estan definidas directamente en `app.js`. No se usan archivos de rutas separados.

Rutas Publicas (sin autenticacion)

Metodo	Ruta	Controlador	Descripcion
GET	`/`	`loginController.index`	Pagina de login
POST	`/login`	`loginController.login`	Procesar login
GET	`/logout`	`loginController.logout`	Cerrar sesion
GET	`/create`	`loginController.createUser`	Formulario registro
POST	`/create`	`loginController.postUser`	Crear cuenta

Barrera de autenticacion

```
app.use(requireAuth);  // Todo lo que sigue requiere sesion activa
```

Rutas Protegidas (requieren sesion)

Home

Metodo	Ruta	Controlador
GET	`/home`	`homeController.index`

Ventas

Metodo	Ruta	Controlador
GET	`/ventas`	`ventasController.index`
GET	`/ventas/cantera`	`ventasController.cantera`
POST	`/ventas/cantera`	`ventasController.finalizarVentaCantera`
GET	`/ventas/cantera/confirmacion`	`ventasController.confirmacionCantera`
GET	`/ventas/viaje`	`ventasController.viaje`
POST	`/ventas/viaje`	`ventasController.crearViajeProgramado`
POST	`/ventas/viaje/finalizar/:id`	`ventasController.finalizarViajeProgramado`
GET	`/ventas/viaje/:id`	`ventasController.detalleViaje`

Transacciones

Metodo	Ruta	Controlador
GET	`/transacciones`	`transaccionesController.index`
GET	`/transacciones/:id`	`transaccionesController.detalle`

Alquileres

Metodo	Ruta	Controlador
GET	`/alquileres`	`alquilerController.index`
GET	`/alquileres/nuevo`	`alquilerController.nuevoAlquiler`
POST	`/alquileres/crear`	`alquilerController.crearAlquiler`
GET	`/alquileres/confirmacion/:id`	`alquilerController.confirmacion`
GET	`/alquileres/detalle/:id`	`alquilerController.detalle`
GET	`/alquileres/editar/:id`	`alquilerController.edicionAlquiler`
POST	`/alquileres/editar/:id`	`alquilerController.guardarEdicion`
POST	`/alquileres/cancelar/:id`	`alquilerController.cancelarAlquiler`
POST	`/alquileres/finalizar/:id`	`alquilerController.finalizarAlquiler`

Contenedores

Metodo	Ruta	Controlador

GET	`/contenedores`	`contenedoresController.index`
GET	`/contenedores/:id`	`contenedoresController.detalle`
GET	`/contenedores/config`	`contenedoresController.config`
POST	`/contenedores/config`	`contenedoresController.guardarConfig`
POST	`/contenedores/crear`	`contenedoresController.crear`
POST	`/contenedores/eliminar/:id`	`contenedoresController.eliminar`

Stock

Metodo	Ruta	Controlador
GET	`/stock`	`stockController.index`
GET	`/stock/nuevo`	`stockController.nuevo`
POST	`/stock/crear`	`stockController.crear`
GET	`/stock/:id`	`stockController.detalle`
GET	`/stock/editar/:id`	`stockController.editar`
POST	`/stock/editar/:id`	`stockController.guardarEdicion`
POST	`/stock/eliminar/:id`	`stockController.eliminar`

Clientes

Metodo	Ruta	Controlador
GET	`/clientes`	`clientesController.index`
GET	`/clientes/nuevo`	`clientesController.nuevo`
POST	`/clientes/crear`	`clientesController.crear`
GET	`/clientes/:id`	`clientesController.detalle`
GET	`/clientes/editar/:id`	`clientesController.editar`
POST	`/clientes/editar/:id`	`clientesController.guardarEdicion`
POST	`/clientes/eliminar/:id`	`clientesController.eliminar`
GET	`/clientes/cuentas`	`clientesController.cuentas`
POST	`/clientes/habilitar-cuenta/:id`	`clientesController.habilitarCuenta`
POST	`/clientes/abonar/:id`	`clientesController.abonar`

API JSON (clientes)

Metodo	Ruta	Controlador
GET	`/api/clientes/buscar`	`clientesController.buscarApi`
POST	`/api/clientes/crear`	`clientesController.crearApi`

Rutas de Administrador (requieren rol admin)

Metodo	Ruta	Controlador
GET	`/configuraciones`	`configController.index`
GET	`/configuraciones/usuarios`	`configController.usuarios`
POST	`/configuraciones/usuarios/crear`	`configController.crearUsuario`
POST	`/configuraciones/usuarios/pausar/:id`	`configController.pausarUsuario`
POST	`/configuraciones/usuarios/eliminar/:id`	`configController.eliminarUsuario`
POST	`/configuraciones/usuarios/cambiar-pass/:id`	`configController.cambiarPassword`

---

6. Controladores

Cada controlador es un objeto que exporta funciones handler de Express `(req, res)`.

login.controller.js

Funciones:

- **`index(req, res)`** — Muestra el formulario de login. Si ya hay sesion activa, redirige a `/home`.
- **`login(req, res)`** — Procesa el login:
  1. Busca el usuario en la base de datos por nombre de usuario
  2. Si la contrasena almacenada es un hash bcrypt (empieza con `$2`), usa `verifyPassword()`
  3. Si es texto plano (usuario legacy), compara directamente y migra a bcrypt automaticamente
  4. Implementa bloqueo por 5 intentos fallidos usando `req.session.loginIntentos`
  5. Si el usuario esta pausado (`activo = false`), rechaza el login
  6. En login exitoso: resetea intentos, guarda `{ id, user, rol }` en sesion
- **`logout(req, res)`** — Destruye la sesion y redirige a `/`.
- **`createUser(req, res)`** / **`postUser(req, res)`** — Formulario de auto-registro. Valida contrasena con `validarContrasena()`, hashea con bcrypt, crea usuario con rol `operador`.

home.controller.js

`index(req, res)` — Dashboard principal. Ejecuta en paralelo (`Promise.all`):

- Alquileres activos (cantidad)
- Contenedores disponibles (cantidad)
- Total de clientes
- Ingreso mensual (suma de transacciones del mes actual)
- Stock bajo (productos con 1-3 unidades)
- Sin stock (productos con 0 unidades)

ventas.controller.js

`index` — Muestra viajes pendientes para hoy y todos los pendientes.

`cantera` — Renderiza formulario de venta en cantera con lista de productos.

`finalizarVentaCantera` — Procesa venta de cantera:

1. Calcula el total del carrito (productos x cantidades)

2. Descuenta stock de cada producto
3. Crea una transaccion de tipo `'Venta Cantera'`
4. Si el metodo de pago es `cuenta_corriente`, agrega movimiento de deuda al cliente

`viaje` / `crearViajeProgramado` — Crea un viaje programado con fecha, productos y cliente. Si `finalizarAhora` esta marcado, crea la transaccion inmediatamente.

`finalizarViajeProgramado` — Finaliza un viaje pendiente: crea transaccion, maneja cuenta corriente.

alquiler.controller.js

`index` — Lista contenedores + alquileres programados + contenedores por finalizar (proximos 2 dias).

`nuevoAlquiler` — Formulario de nuevo alquiler. Soporta pre-llenado via `?renovar=id` para renovaciones.

`crearAlquiler` — Logica clave:

- Si el contenedor ya esta `Alquilado`: crea alquiler con estado `programado` (cola)
- Si esta `Disponible`: crea alquiler `activo` y actualiza el contenedor

`finalizarAlquiler` — Flujo completo:

1. Crea transaccion con el monto del alquiler
2. Si es `cuenta_corriente`, agrega deuda al cliente
3. Libera el contenedor (estado `Disponible`)
4. Si hay un alquiler `programado` en cola, lo activa automaticamente

`cancelarAlquiler` — Libera el contenedor sin crear transaccion (cancelacion sin cobro).

contenedores.controller.js

CRUD de contenedores + configuracion global de precios (`precio_dia` y `precio_alquiler`) que se aplica a todos los contenedores disponibles.

stock.controller.js

CRUD completo de productos: listar, crear, editar, eliminar, ver detalle.

clientes.controller.js

CRUD de clientes + sistema de cuenta corriente:

- **`cuentas`** — Lista clientes con cuenta corriente habilitada y sus movimientos
- **`habilitarCuenta`** — Habilita cuenta corriente para un cliente
- **`abonar`** — Registra un pago (abono) contra la deuda del cliente. El monto maximo es el saldo deudor actual
- **`buscarApi` / `crearApi`** — Endpoints JSON para busqueda/creacion AJAX desde formularios de ventas y alquileres

transacciones.controller.js

- **`index`** — Listado con filtros multiples: id, tipo, cliente, rango de fechas, rango de montos
- **`detalle`** — Detalle de una transaccion individual

config.controller.js

Administracion de usuarios del sistema (solo accesible con rol `admin`):

- Listar usuarios
- Crear usuario (con validacion de contrasena y hasheo bcrypt)
- Pausar/activar usuario (toggle booleano)
- Eliminar usuario
- Cambiar contrasena (con validacion y hasheo)

---

7. Capa de Datos (Store)

Todos los archivos `store/` siguen el mismo patron:

1. Importan el cliente Supabase de `lib/supabase.js`
2. Definen una funcion `mapRow(row)` que convierte columnas `snake_case` de PostgreSQL a propiedades `camelCase` de JavaScript
3. Exportan funciones async que ejecutan queries contra Supabase

Convencion de mapeo snake_case a camelCase

Cada store tiene su propia funcion `mapRow()` especifica para su tabla. Ejemplo tipico:

```
function mapRow(r) {
    return {
        id: r.id,
        clienteId: r.cliente_id,
        clienteNombre: r.cliente_nombre,
        inicioAlquiler: r.inicio_alquiler,
        // ... etc
    };
}
```

dbAlquiler.js — Tabla: `alquileres`

Funcion	Query	Retorno
`crearAlquiler(datos)`	INSERT	Alquiler creado
`listAlquileres()`	SELECT * ORDER BY id DESC	Array
`alquilerById(id)`	SELECT WHERE id	Objeto o null
`alquileresProgramados()`	SELECT WHERE estado='programado'	Array
`alquileresActivos()`	SELECT WHERE estado='activo'	Array
`alquileresProgramadosPorContenedor(id)`	SELECT WHERE contenedor_id AND estado='programado' LIMIT 1	Objeto o null
`activarAlquiler(id)`	UPDATE estado='activo'	void
`finalizarAlquilerRecord(id)`	UPDATE estado='finalizado'	void

Estados de alquiler: `activo`, `programado`, `finalizado`, `cancelado`

dbClientes.js — Tablas: `clientes`, `movimientos_cuenta`

Funcion	Descripcion
`listClientes()`	Todos los clientes
`clienteById(id)`	Cliente por ID
`buscarClientes({ id, dni, nombre })`	Busqueda flexible (ilike en nombre/apellido)
`crearCliente(datos)`	Crea cliente con saldo=0

`editarCliente(id, datos)`	Actualizacion parcial
`eliminarCliente(id)`	Eliminacion
`habilitarCuentaCorriente(id)`	Activa cuenta_corriente=true
`agregarMovimiento(id, { tipo, descripcion, monto })`	INSERT en movimientos_cuenta + actualiza saldo
`abonarCuenta(id, monto)`	Wrapper: agregarMovimiento con tipo='pago'
`clientesConCuenta()`	Clientes con cuenta corriente + sus movimientos
`nombreCompleto(c)`	Helper: retorna "nombre apellido"

dbContenedor.js — Tabla: `contenedores`

Funcion	Descripcion
`listContenedores()`	Todos los contenedores
`contenedorById(id)`	Por ID
`listContenedoresAlquilados()`	WHERE estado='Alquilado'
`listContenedoresDisponibles()`	WHERE estado='Disponible'
`listContenedoresPorFinalizar()`	Alquilados con fin_alquiler entre hoy y +2 dias
`actualizarContenedor(id, datos)`	UPDATE parcial
`actualizarPrecios(precioDia, precioAlquiler)`	UPDATE masivo en todos los Disponibles
`finalizarAlquiler(id)`	Reset a estado Disponible con precios default
`crearContenedor()`	INSERT copiando precios de un existente
`eliminarContenedor(id)`	DELETE solo si no esta Alquilado

Estados de contenedor: `Disponible`, `Alquilado`

dbStock.js — Tabla: `stock`

Funcion	Descripcion
`listProds()`	Todos los productos
`prodsById(id)`	Por ID
`crearProducto({ producto, precio, stock })`	INSERT
`actualizarProducto(id, datos)`	UPDATE parcial
`eliminarProducto(id)`	DELETE

dbTransacciones.js — Tabla: `transacciones`

Funcion	Descripcion
`crearTransaccion({ tipo, clienteId, cliente, monto, descripcion, metodoPago })`	INSERT
`listTransacciones()`	SELECT * ORDER BY id DESC

`filtrarTransacciones(filters)`	Filtro flexible: id, tipo, cliente, fechas, montos

Tipos de transaccion: `'Alquiler'`, `'Venta Cantera'`, `'Venta Viaje'`

dbUsers.js — Tabla: `usuarios`

Funcion	Descripcion
`listUsuarios()`	Todos los usuarios
`usuarioById(id)`	Por ID
`buscarPorUser(user)`	Por nombre de usuario
`crearUsuario({ nombre, apellido, user, password, rol })`	INSERT
`actualizarUsuario(id, datos)`	UPDATE generico
`eliminarUsuario(id)`	DELETE
`pausarUsuario(id)`	Toggle del campo `activo`
`resetPassword(id, newPassword)`	UPDATE del campo password

Roles: `'admin'`, `'operador'`

dbViajes.js — Tabla: `viajes`

Funcion	Descripcion
`crearViaje(datos)`	INSERT
`listViajes()`	SELECT * ORDER BY id DESC
`viajeById(id)`	Por ID
`finalizarViaje(id)`	UPDATE estado='finalizado'
`viajesPendientesHoy()`	WHERE estado='pendiente' AND fecha=hoy
`viajesPendientes()`	WHERE estado='pendiente' ORDER BY fecha ASC

Estados de viaje: `'pendiente'`, `'finalizado'`

---

8. Biblioteca de Utilidades (lib/)

lib/supabase.js

Crea y exporta una instancia unica (singleton) del cliente Supabase:

```
const { createClient } = require('@supabase/supabase-js');
require('dotenv').config();

const supabase = createClient(
    process.env.SUPABASE_URL,
    process.env.SUPABASE_KEY
);
```

```
module.exports = supabase;
```

Todos los archivos `store/` importan esta instancia.

lib/password.js

Tres funciones de utilidad para manejo de contrasenas:

```
const bcrypt = require('bcrypt');
const SALT_ROUNDS = 10;

// Regex: min 8 chars, mayuscula, minuscula, numero, caracter especial
const PASSWORD_REGEX = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[+\-*/?&%$@!#^_=<>|~]).{8,}$/;
```

Funcion	Descripcion
`validarContrasena(password)`	Retorna `null` si es valida, o un string con el error
`hashPassword(password)`	Retorna hash bcrypt (10 salt rounds)
`verifyPassword(password, hash)`	Retorna `true/false`

---

9. Middleware de Autenticacion

middleware/requireAuth.js

```
module.exports = function (req, res, next) {
    if (!req.session?.user) return res.redirect('/');
    next();
};
```

Se aplica **globalmente** en `app.js` despues de las rutas publicas. Todo lo definido despues de `app.use(requireAuth)` requiere sesion.

middleware/requireAdmin.js

```
module.exports = function (req, res, next) {
    if (req.session?.user?.rol !== 'admin') {
        return res.status(403).render('error', {
            message: 'Acceso denegado',
            error: { status: 403, stack: '' }
        });
    }
    next();
};
```

Se aplica **por ruta** en las rutas de configuracion:

```
app.get('/configuraciones', requireAdmin, configController.index);
```

---

10. Vistas (EJS)

El motor de plantillas es EJS (Embedded JavaScript). Las vistas estan en `views/`.

Estructura de vistas

Carpeta	Vistas	Descripcion
`views/`	`home.ejs`, `error.ejs`	Dashboard y pagina de error
`views/login/`	`index.ejs`, `create.ejs`	Login y registro
`views/alquileres/`	`index`, `nuevo_alquiler`, `confirmacion`, `detalle`, `editar`	Gestion de alquileres
`views/clientes/`	`index`, `nuevo_cliente`, `detalle`, `editar`, `cuentas`	Gestion de clientes
`views/configuraciones/`	`index`, `usuarios`	Configuracion (admin)
`views/contenedores/`	`index`, `detalle`, `config`	Gestion de contenedores
`views/stock/`	`index`, `nuevo_stock`, `detalle`, `editar`	Gestion de stock
`views/transacciones/`	`index`, `detalle`	Historial financiero
`views/ventas/`	`index`, `cantera`, `viaje`, `confirmacion`, `detalle_viaje`	Ventas

Parciales reutilizables (views/partials/)

Parcial	Uso
`sidebar.ejs`	Navegacion lateral (incluida en todas las paginas)
`nav.ejs`	Barra de navegacion superior
`scripts.ejs`	Tags de scripts comunes
`buscar_cliente.ejs`	Widget AJAX de busqueda de clientes
`form_alquiler.ejs`	Campos del formulario de alquiler
`form_venta_cantera.ejs`	Campos del formulario de venta cantera
`form_venta_viaje.ejs`	Campos del formulario de venta viaje
`metodo_pago.ejs`	Selector de metodo de pago

Variables globales disponibles en todas las vistas

- `sessionUser` — Objeto `{ id, user, rol }` del usuario logueado (o `undefined`)
- `formatFecha(dateStr)` — Funcion que convierte fecha ISO a `DD-MM-YYYY`

Inclusion de parciales

```
<%- include('../partials/sidebar.ejs') %>
```

El parcial `metodo_pago.ejs` genera un `<select>` con las opciones:

- `efectivo` (default)
- `transferencia`
- `cuenta_corriente` (solo visible si el cliente tiene cuenta corriente habilitada)

11. Assets Estaticos (CSS/SCSS/JS)

SCSS

El sistema de estilos usa SCSS con compilacion via `sass --watch`. El archivo principal es `public/scss/style.scss` que importa 18 parciales:

```
@use "sass:color";
@import 'variables';
@import 'reset';
@import 'layout';
@import 'sidebar';
@import 'main';
// ... etc
```

Compilacion: `sass --watch public/scss/style.scss:public/css/styles.css`

El CSS compilado se sirve desde `public/css/styles.css`.

JavaScript del cliente (public/javascript/)

Archivo	Descripcion
`sidebar.js`	Toggle del sidebar responsive
`formValidation.js`	Validacion generica de formularios (campos requeridos)
`buscarCliente.js`	AJAX para buscar/crear clientes desde formularios
`ventas.js`	Logica compartida de ventas
`ventasCantera.js`	Carrito de venta cantera (agregar/quitar productos, calcular total)
`ventasViaje.js`	Formulario de venta viaje
`contenedores.js`	Interacciones de la vista de contenedores
`alquilerService.js`	Funciones del formulario de alquiler

12. Base de Datos — Tablas y Relaciones

La base de datos esta alojada en Supabase (PostgreSQL). Todas las tablas usan `snake_case`.

Tabla: `usuarios`

Columna	Tipo	Descripcion
`id`	serial PK	ID autoincremental
`nombre`	text	Nombre del usuario
`apellido`	text	Apellido
`user`	text UNIQUE	Nombre de usuario (login)
`password`	text	Hash bcrypt de la contrasena
`rol`	text	`'admin'` o `'operador'`

`activo`	boolean	`true` = activo, `false` = pausado

Tabla: `contenedores`

Columna	Tipo	Descripcion
`id`	serial PK	ID del contenedor
`estado`	text	`'Disponible'` O `'Alquilado'`
`cliente_id`	integer FK	Referencia a `clientes.id`
`cliente`	text	Nombre del cliente (desnormalizado)
`inicio_alquiler`	text/date	Fecha inicio del alquiler activo
`fin_alquiler`	text/date	Fecha fin del alquiler activo
`direccion_alquiler`	text	Direccion de entrega
`precio_dia`	numeric	Precio por dia (configuracion)
`precio_alquiler`	numeric	Precio del alquiler actual
`metodo_pago`	text	`'efectivo'` O `'transferencia'`

Tabla: `alquileres`

Columna	Tipo	Descripcion
`id`	serial PK	ID del alquiler
`contenedor_id`	integer FK	Referencia a `contenedores.id`
`cliente_id`	integer FK	Referencia a `clientes.id`
`cliente_nombre`	text	Nombre del cliente
`inicio_alquiler`	text/date	Fecha inicio
`fin_alquiler`	text/date	Fecha fin
`direccion_alquiler`	text	Direccion
`precio_alquiler`	numeric	Precio
`metodo_pago`	text	CHECK: `'efectivo'` O `'transferencia'`
`estado`	text	`'activo'`, `'programado'`, `'finalizado'`, `'cancelado'`

Tabla: `clientes`

Columna	Tipo	Descripcion
`id`	serial PK	ID del cliente
`nombre`	text	Nombre
`apellido`	text	Apellido
`dni`	text	Documento de identidad

`telefono`	text	Telefono
`email`	text	Email
`direccion`	text	Direccion
`cuenta_corriente`	boolean	Si tiene cuenta corriente habilitada
`saldo`	numeric	Saldo de cuenta corriente (negativo = deuda)

Tabla: `movimientos_cuenta`

Columna	Tipo	Descripcion
`id`	serial PK	ID del movimiento
`cliente_id`	integer FK	Referencia a `clientes.id`
`tipo`	text	`'deuda'` o `'pago'`
`descripcion`	text	Descripcion del movimiento
`monto`	numeric	Monto (negativo = deuda, positivo = pago)
`created_at`	timestamp	Fecha de creacion

Tabla: `stock`

Columna	Tipo	Descripcion
`id`	serial PK	ID del producto
`producto`	text	Nombre del producto
`precio`	numeric	Precio unitario
`stock`	integer	Cantidad en stock

Tabla: `transacciones`

Columna	Tipo	Descripcion
`id`	serial PK	ID de la transaccion
`tipo`	text	`'Alquiler'`, `'Venta Cantera'`, `'Venta Viaje'`
`cliente_id`	integer FK	Referencia a `clientes.id`
`cliente`	text	Nombre del cliente
`monto`	numeric	Monto total
`descripcion`	text	Descripcion detallada
`metodo_pago`	text	Metodo de pago utilizado
`created_at`	timestamp	Fecha de creacion

Tabla: `viajes`

Columna	Tipo	Descripcion

`id`	serial PK	ID del viaje
`cliente_id`	integer FK	Referencia a `clientes.id`
`cliente`	text	Nombre del cliente
`fecha`	text/date	Fecha programada
`productos`	jsonb	Array de productos con cantidades
`total`	numeric	Monto total
`metodo_pago`	text	Metodo de pago
`estado`	text	`'pendiente'` O `'finalizado'`
`direccion`	text	Direccion de entrega

Diagrama de Relaciones

```
usuarios (standalone)

clientes ---< movimientos_cuenta
    |
    +---< contenedores (cliente_id)
    |
    +---< alquileres (cliente_id)
    |
    +---< transacciones (cliente_id)
    |
    +---< viajes (cliente_id)

contenedores ---< alquileres (contenedor_id)

stock (standalone, referenciado en viajes.productos como JSON)
```

---

13. Flujos de Negocio

Flujo de Alquiler

```
1. Usuario accede a /alquileres/nuevo
2. Selecciona contenedor disponible (o por finalizar para renovacion)
3. Selecciona/crea cliente
4. Define fechas, direccion y precio
5. POST /alquileres/crear
   |
   +-- Contenedor Disponible:
   |     - Crea alquiler con estado 'activo'
   |     - Actualiza contenedor a estado 'Alquilado'
   |
   +-- Contenedor Alquilado (renovacion):
         - Crea alquiler con estado 'programado'
         - El contenedor mantiene su estado actual
6. Al finalizar (POST /alquileres/finalizar/:id):
   - Crea transaccion con el monto
   - Si es cuenta_corriente: agrega deuda al cliente
```

```
   - Libera contenedor (estado Disponible)
   - Si hay alquiler programado en cola: lo activa automaticamente
```

Flujo de Venta Cantera

```
1. Usuario accede a /ventas/cantera
2. Agrega productos al carrito (JS del cliente)
3. Selecciona cliente y metodo de pago
4. POST /ventas/cantera
   - Descuenta stock de cada producto
   - Crea transaccion tipo 'Venta Cantera'
   - Si es cuenta_corriente: agrega deuda al cliente
5. Redirige a confirmacion
```

Flujo de Venta Viaje

```
1. Usuario accede a /ventas/viaje
2. Completa datos: cliente, productos, fecha, direccion
3. POST /ventas/viaje
   |
   +-- finalizarAhora = true:
   |     - Crea viaje con estado 'finalizado'
   |     - Crea transaccion inmediatamente
   |
   +-- finalizarAhora = false:
         - Crea viaje con estado 'pendiente'
         - Aparece en el listado de viajes pendientes
4. Para viajes pendientes: POST /ventas/viaje/finalizar/:id
   - Crea transaccion
   - Cambia estado a 'finalizado'
```

Flujo de Cuenta Corriente

```
1. Admin habilita cuenta corriente para un cliente
2. El cliente puede pagar con metodo 'cuenta_corriente'
3. Cada operacion con cuenta_corriente genera:
   - Una transaccion normal (registro financiero)
   - Un movimiento de deuda en movimientos_cuenta (saldo negativo)
4. El admin puede registrar abonos (pagos parciales)
   - Agrega movimiento tipo 'pago' (saldo positivo)
   - El monto maximo es el saldo deudor actual
```

Flujo de Autenticacion

```
1. GET / -> Formulario de login
2. POST /login
   - Busca usuario por nombre
   - Verifica contrasena (bcrypt o texto plano legacy)
   - Si es texto plano y coincide: migra a bcrypt automaticamente
   - Maximo 5 intentos fallidos por sesion
   - En exito: guarda { id, user, rol } en sesion
3. Todas las rutas posteriores a requireAuth verifican sesion
4. Rutas /configuraciones/* verifican ademas rol='admin'
5. GET /logout -> destruye sesion
```

---

14. Despliegue (Render.com)

El archivo `render.yaml` define el servicio:

```yaml
services:
  - type: web
    name: seminario-gestion
    runtime: node
    buildCommand: npm install
    startCommand: npm start
    plan: free
    envVars:
      - key: NODE_ENV
        value: production
      - key: SUPABASE_URL
        sync: false
      - key: SUPABASE_KEY
        sync: false
```

Variables de entorno requeridas en el dashboard de Render:

- `SUPABASE_URL` — URL del proyecto Supabase
- `SUPABASE_KEY` — Clave del proyecto Supabase

Estas variables estan marcadas como `sync: false` porque son secretas y deben configurarse manualmente.

Nota: El SCSS se compila localmente y el CSS resultante (`public/css/styles.css`) se versiona en git. No hay paso de build para SCSS en produccion.

---

15. Seguridad

Autenticacion

- Sesiones con `express-session` (cookie de 8 horas)
- Contrasenas hasheadas con **bcrypt** (10 salt rounds)
- Migracion automatica de contrasenas legacy en texto plano
- Bloqueo tras **5 intentos fallidos** (por sesion)
- Usuarios pueden ser **pausados** por el admin (impide login)

Validacion de contrasena

Requisitos minimos (aplicados en creacion y cambio):

- Minimo 8 caracteres
- Al menos una mayuscula
- Al menos una minuscula
- Al menos un numero
- Al menos un caracter especial (`+` , `-` , `*` , `/` , `?` , `&` , `%` , `$` , `@` , `!` , `#` , etc.)

Autorizacion

- **requireAuth**: Verifica sesion activa en todas las rutas protegidas
- **requireAdmin**: Verifica rol `admin` en rutas de configuracion
- Separacion de roles: `admin` y `operador`

Consideraciones

- El secreto de sesion esta hardcodeado en `app.js` — se recomienda moverlo a `.env`

- No hay proteccion CSRF en formularios
- No hay rate limiting a nivel de servidor (solo por sesion)
- Las credenciales de Supabase se manejan exclusivamente via `.env` y nunca se versionan

---

Manual generado para el proyecto Sistema de Gestion Empresarial — Suelo Sur SAS